Machine Learning Exam Questions

Luu Minh Sao Khue

December 29, 2025

1. Machine Learning Fundamentals and Evaluation Metrics

1.1 What is Machine Learning and how does it differ from traditional programming?

Machine Learning is a paradigm in which models learn patterns and relationships from data rather than being explicitly programmed with fixed rules. Instead of writing logic by hand, the practitioner defines a model structure and a learning objective, and the model adapts its parameters based on observed examples.

1.2 What is the difference between supervised and unsupervised learning?

In supervised learning, models are trained using labeled data, where each input is associated with a known target output. In unsupervised learning, no labels are provided, and the goal is to discover underlying structure or patterns in the data, such as clusters or low-dimensional representations.

1.3 What is the difference between regression and classification tasks?

Regression aims to predict a continuous numerical value, such as price or temperature, while classification aims to assign inputs to discrete categories or class labels. The nature of the target variable determines which type of task is appropriate.

1.4 Explain the difference between MAE and MSE in regression evaluation.

Mean Absolute Error (MAE) measures the average absolute difference between predicted and true values, treating all errors equally. The unit of MAE is the same as the target. Mean Squared Error (MSE) squares the errors before averaging, which penalizes large errors more strongly and makes the metric more sensitive to outliers. The unit of MSE is squared unit of the target.

1.5 What is accuracy and when is it a suitable metric?

Accuracy is the proportion of correctly classified samples among all samples. It is suitable when classes are balanced and when all types of errors have similar importance.

1.6 Why can accuracy be misleading in imbalanced classification problems?

In imbalanced datasets, a model can achieve high accuracy by predicting the majority class most of the time while failing to correctly identify minority classes. This makes accuracy insufficient for evaluating real performance.

1.7 In what situations is recall more important than precision?

Recall is more important when missing positive cases is costly, such as in medical diagnosis or fraud detection. In these scenarios, identifying all positive cases is prioritized over avoiding false alarms.

2. Linear Regression, Ridge, and Lasso

2.1 Explain the idea of Linear Regression.

Linear Regression models the relationship between input features and a target variable as a linear combination of the features. The model assumes that each feature contributes independently and additively to the prediction, and learning consists of finding the coefficients that best fit the observed data. By minimizing the discrepancy between predicted and true values, Linear Regression provides a simple, interpretable baseline for understanding how variables influence the outcome.

2.2 Why can Linear Regression perform poorly when features are highly correlated?

When features are highly correlated, the model may assign unstable or extreme coefficients to them. This multicollinearity makes the solution sensitive to small changes in the data and can reduce interpretability and generalization.

2.3 What problem does Ridge Regression aim to solve compared to Linear Regression?

Ridge Regression addresses the instability of Linear Regression by penalizing large coefficients. By adding an ℓ_2 regularization term to the loss function, it shrinks coefficients toward zero, reducing variance and improving generalization.

2.4 How does Lasso Regression differ conceptually from Ridge Regression?

Lasso Regression uses an ℓ_1 penalty, which encourages sparsity in the model. As a result, Lasso can drive some coefficients exactly to zero, effectively performing feature selection, whereas Ridge typically keeps all features with small but nonzero coefficients.

2.5 Why is there a trade-off when choosing the regularization strength in Ridge or Lasso?

Increasing regularization reduces model variance but increases bias by restricting the flexibility of the model. Choosing the regularization strength therefore requires balancing underfitting and overfitting to achieve optimal performance.

3. Logistic Regression and Multilayer Perceptron (MLP)

3.1 Why is Logistic Regression considered a linear model even though it uses a nonlinear function?

Logistic Regression is linear because it models a linear combination of input features before applying the sigmoid function. The nonlinearity only maps this linear score to a probability, but the decision boundary itself remains linear in the feature space.

3.2 What does the output of Logistic Regression represent probabilistically?

The output of Logistic Regression represents the estimated probability that a sample belongs to the positive class. This probabilistic interpretation makes the model suitable for decision-making tasks where confidence or risk estimation is important.

3.3 Why can Logistic Regression fail on complex real-world problems?

Logistic Regression assumes a linear decision boundary and cannot capture complex nonlinear relationships or feature interactions unless these are explicitly added through feature engineering. As a result, its expressive power is limited.

3.4 How does a Multilayer Perceptron overcome the limitations of Logistic Regression?

An MLP introduces hidden layers with nonlinear activation functions, allowing it to learn nonlinear decision boundaries and complex feature interactions automatically. This makes MLPs significantly more expressive than linear models.

4. Decision Trees

4.1 What is the main idea behind a Decision Tree model?

A Decision Tree models decision-making as a sequence of hierarchical rules. At each internal node, the data are split according to a feature-based condition, and this process continues until a prediction is made at a leaf node. The model recursively partitions the feature space into regions with similar target values.

4.2 Why are Decision Trees considered highly interpretable models?

Decision Trees are interpretable because their structure mirrors human decision-making. Each prediction can be explained by following a path from the root to a leaf, revealing which conditions were used and in what order.

4.3 What role do impurity measures play in training a Decision Tree?

Impurity measures quantify how mixed the target values are within a node. During training, the tree selects splits that most reduce impurity, guiding the model toward partitions where samples are more homogeneous with respect to the target.

4.4 Why do Decision Trees tend to overfit when allowed to grow deep?

As a tree grows deeper, it creates increasingly specific rules that may fit noise in the training data rather than general patterns. This leads to low training error but poor generalization to unseen data.

4.5 Why are Decision Trees considered high-variance models?

Decision Trees are sensitive to small changes in the training data, which can result in very different tree structures. This instability leads to high variance, making individual trees prone to overfitting unless regularized or combined in an ensemble.

5. Bagging and Random Forests

5.1 What is the main idea behind bagging (bootstrap aggregating)?

Bagging reduces model variance by training multiple models independently on different bootstrap samples of the training data and averaging their predictions. By combining many high-variance models, bagging produces a more stable and robust predictor.

5.2 How do Bagging and Random Forest differ from Boosting methods conceptually?

Bagging and Random Forest focus on variance reduction by averaging independent models, while boosting methods focus on bias reduction by sequentially correcting errors. This leads to different strengths: bagging is more robust to noise, whereas boosting can achieve higher accuracy on complex but clean datasets.

5.3 How does Random Forest extend the idea of bagging?

Random Forest builds on bagging by introducing additional randomness in feature selection. At each split, only a random subset of features is considered, which decorrelates the trees and further improves the variance reduction achieved by averaging.

6. Boosting and AdaBoost

6.1 What is the general idea of boosting in ensemble learning?

Boosting is an ensemble learning strategy that combines multiple weak learners to form a strong predictor. Instead of training models independently, boosting trains them sequentially, with each learner focusing more on the instances that were mispredicted by the previous ones.

6.2 How does AdaBoost differ from bagging-based methods such as Random Forests?

AdaBoost emphasizes hard-to-classify samples by increasing their importance during training, while bagging methods train models independently on randomly resampled

datasets. As a result, AdaBoost primarily reduces bias, whereas bagging mainly reduces variance.

6.3 How does AdaBoost assign importance to training samples?

AdaBoost maintains a weight for each training sample. After each weak learner is trained, the weights of misclassified samples are increased, while the weights of correctly classified samples are decreased. This forces subsequent learners to focus more on difficult cases.

7. Gradient Boosting Trees

7.1 What are Gradient Boosting Trees?

Gradient Boosting Trees are an ensemble learning method that builds a strong predictive model by sequentially combining many weak learners, most commonly shallow decision trees. Each new tree is added to improve the performance of the current ensemble by correcting its errors. The main intuition is that many small, incremental improvements can produce a highly accurate model. Instead of fitting a single complex model, Gradient Boosting starts with a simple predictor and improves it step by step, with each new tree addressing what the model still gets wrong.

7.2 How does the sequential learning process work in Gradient Boosting?

The algorithm begins with an initial prediction, often a constant value. At each iteration, it computes how far the current predictions are from the true target values. A new decision tree is trained to reduce these errors, and its predictions are added to the existing model. This process repeats until the model converges or a stopping criterion is reached.

7.3 What does the "gradient" represent in Gradient Boosting Trees?

In Gradient Boosting, the gradient represents the direction in which the model predictions should be adjusted to reduce the loss function most effectively. At each iteration, the negative gradient of the loss with respect to the current predictions is computed for every training example. A new decision tree is then trained to approximate this negative gradient, allowing the ensemble to perform a step of gradient descent in function space rather than in parameter space.

7.4 Why can Gradient Boosting be viewed as gradient descent in function space?

Instead of optimizing parameters directly, Gradient Boosting incrementally builds a function by adding new trees. Each tree represents a step in the direction that reduces the loss most rapidly, analogous to gradient descent, but performed over functions rather than numeric parameters.

7.5 Why are decision trees typically used as base learners?

Decision trees are flexible models that can capture nonlinear relationships and feature interactions. When kept shallow, they act as weak learners that are easy to control and

regularize. This makes them well suited for the incremental, error-correcting nature of Gradient Boosting.

7.6 What is the role of the learning rate in Gradient Boosting?

The learning rate controls how much each new tree contributes to the overall model. Smaller learning rates lead to slower learning but often improve generalization when combined with a larger number of trees.

7.7 What are the main strengths of Gradient Boosting Trees?

Gradient Boosting Trees perform particularly well on tabular data with complex, non-linear relationships. They require little feature engineering, handle mixed feature types naturally, and often achieve high predictive accuracy in practice.

7.8 What are the main limitations of Gradient Boosting Trees?

Gradient Boosting Trees are sensitive to noise and outliers, as later trees focus on correcting difficult cases. Without proper regularization, the model may overfit. Training is also computationally more expensive due to the sequential nature of the algorithm.

8. Support Vector Machine (SVM)

8.1 What is the main objective of a Support Vector Machine (SVM)?

The objective of SVM is to find a decision boundary that separates classes while maximizing the margin between them. The margin is defined as the distance between the separating hyperplane and the closest data points from each class. Maximizing this margin leads to better generalization and reduces the risk of overfitting.

8.2 What are support vectors and why are they important?

Support vectors are the data points that lie closest to the decision boundary. They uniquely determine the position and orientation of the separating hyperplane. Removing non-support-vector points does not affect the final solution, whereas changing a support vector alters the classifier.

8.3 How does SVM handle non-linearly separable data?

When data are not linearly separable, SVM uses kernel functions to implicitly map the input features into a higher-dimensional space. In this transformed space, a linear separating hyperplane can often be found even though the separation is nonlinear in the original feature space.

8.4 Why are kernel functions used in SVM?

Kernel functions allow SVM to construct nonlinear decision boundaries without explicitly computing the high-dimensional feature mapping. This approach, known as the kernel trick, enables efficient computation while retaining expressive decision surfaces.

8.5 What are the main limitations of SVM?

SVM performance is sensitive to the choice of kernel and hyperparameters, such as the regularization parameter and kernel-specific settings. Training can be computationally expensive for large datasets, and the resulting models are often less interpretable than simple linear classifiers.

9. k-Means Clustering

9.1 Why does k-Means require the number of clusters to be specified in advance?

The algorithm optimizes cluster assignments with respect to a fixed number of centroids. Since both the assignment step and centroid update depend on k, different values of k result in different optimization problems and different solutions.

9.2 How does the k-Means algorithm work?

The algorithm alternates between two steps: assigning each data point to the nearest centroid and updating each centroid as the mean of its assigned points. This process is repeated until assignments stabilize or a convergence criterion is met.

9.3 Why is k-Means sensitive to initialization?

Because k-Means uses iterative optimization, it may converge to local minima. Different initial centroid positions can lead to different final clusterings, and poor initialization may result in suboptimal solutions.

9.4 What assumptions does k-Means make?

k-Means assumes clusters are roughly spherical, have similar sizes and densities, and can be well separated using Euclidean distance. When these assumptions are violated, clustering quality degrades.

9.5 What are the main limitations of k-Means?

The algorithm performs poorly on non-spherical clusters, is sensitive to outliers, and struggles when clusters vary significantly in size or density.

10. Hierarchical Clustering

10.1 What is hierarchical clustering?

Hierarchical clustering builds a nested structure of clusters represented as a tree called a dendrogram. This structure allows analysis of the data at multiple levels of granularity.

10.2 What is the difference between agglomerative and divisive clustering?

Agglomerative clustering starts with each data point as its own cluster and iteratively merges clusters, while divisive clustering starts with all points in one cluster and recursively splits them. Agglomerative methods are more commonly used due to their simpler implementation.

10.3 What is linkage and why is it important?

Linkage defines how distances between clusters are computed. It directly determines which clusters are merged at each step and strongly influences the resulting cluster hierarchy.

10.4 How is a dendrogram interpreted?

The dendrogram visualizes the order and distance at which clusters merge. Cutting the tree at a chosen height yields a specific number of clusters.

11. DBSCAN

11.1 How does DBSCAN define a cluster?

DBSCAN defines a cluster as a dense region of points connected through neighborhood relationships. Clusters can take arbitrary shapes and are separated by regions of low density.

11.2 What roles do `eps` and `minPts` play?

- `eps`: radius defining the local neighborhood.
- `minPts`: minimum number of neighbors required to form a dense region.

11.3 Types of points in DBSCAN

- Core points: have at least `minPts` neighbors within `eps`.
- Border points: reachable from a core point but not dense themselves.
- Noise points: not reachable from any core point.

11.4 What are the main limitations of DBSCAN?

DBSCAN is sensitive to parameter selection, struggles with datasets containing clusters of varying density, and becomes less effective in high-dimensional spaces where distance measures lose meaning.

12. Principal Component Analysis (PCA)

12.1 What problem does PCA solve?

PCA reduces dimensionality by transforming correlated features into a smaller set of uncorrelated components while preserving as much variance as possible.

12.2 What is a principal component?

A principal component is a direction in feature space defined as a linear combination of the original variables. Components are orthogonal and ordered by the amount of variance they explain.

12.3 How is the first principal component determined?

The first principal component is the direction that maximizes the variance of the projected data. It corresponds to the eigenvector associated with the largest eigenvalue of the covariance matrix.

12.4 Why must principal components be orthogonal?

Orthogonality ensures that each component captures new, non-redundant information and that variance is not counted multiple times.

12.5 Why do we center and sometimes scale data before PCA?

Centering ensures variance is measured relative to the data mean. Scaling is necessary when features have different units or magnitudes, preventing dominant features from biasing the components.

12.6 What are the limitations of PCA?

PCA captures only linear relationships, is sensitive to outliers, and often produces components that are difficult to interpret physically.

13. t-SNE

13.1 What is t-SNE used for?

t-SNE is primarily used for visualizing high-dimensional data in two or three dimensions. It emphasizes preservation of local neighborhood structure rather than global geometry.

13.2 Is t-SNE dimensionality reduction or visualization?

Although t-SNE technically performs dimensionality reduction, it is mainly a visualization tool. The resulting embeddings are not suitable for downstream modeling or reuse.

13.3 How does t-SNE differ from PCA?

PCA preserves global variance through linear projections, while t-SNE is a nonlinear method that focuses on maintaining local similarities between nearby points.

13.4 What is perplexity in t-SNE?

Perplexity controls the effective neighborhood size used to compute similarities. Lower values emphasize local structure, while higher values incorporate more global information.

13.5 What objective function does t-SNE optimize?

t-SNE minimizes the Kullback–Leibler divergence between pairwise similarity distributions in the original space and the low-dimensional embedding, heavily penalizing errors involving nearby points.